

Chat harness architecture

The harness is the server-side control plane around report chat. It is not the Gemini model, and it is not the PDF search index. It decides intent, retrieval shape, which tools exist on each step, and whether a drafted fact is allowed to persist.

Mental model. The LLM proposes. The harness constrains. The engineer Applies. There is no commit edit path: live report chat always writes `ai_fix` comments plus red/green marks. Nothing lands in `report_sections` until Apply / Dismiss.

1. Three layers that people mix up

A. Harness (this briefing)

`src/lib/ai/chat/*` · `POST /api/reports/[reportId]/chat` · intent, TurnPlan, prepareStep, grounding, compaction

B. Retrieval (briefing 3 writes the index · briefing 4 queries it)

Same Postgres: pgvector HNSW + English FTS — not Pinecone, not BM25. Ingest is Vertex-only.

C. Orchestrator model

Gemini 3.7 Flash, thinking medium · page extracts: 3.5 Flash-Lite · the model barely writes; it mostly re-reads context

Figure 1. Chat stays the control plane. Do not replace agentic grep with a standalone search product. How files become rows: briefing 3. How search reads those rows: briefing 4.

What the harness owns

- Latest-turn intent: social | read | write
- Retrieval policy: focused | adaptive | comprehensive
- Per-step activeTools / optional toolChoice
- Citation ledger + groundDraftText
- In-turn message compaction and tool-result budgets
- 270s wall-clock abort (not a tool-step cap)

What it does not own

- TipTap section JSON (autosave PATCH)
- Criteria eval / suggestion generation
- PDF bytes / Vertex ingest / embeddings
- Analytics worksheet (separate `/analytics/chat`, same ChatPanel)
- Apply/Dismiss merge (`src/lib/suggestions`)

2. Request lifecycle

Entry point: `src/app/api/reports/[reportId]/chat/route.ts`. One HTTP POST is one *turn*. A turn may contain many *steps* (model → tools → model).

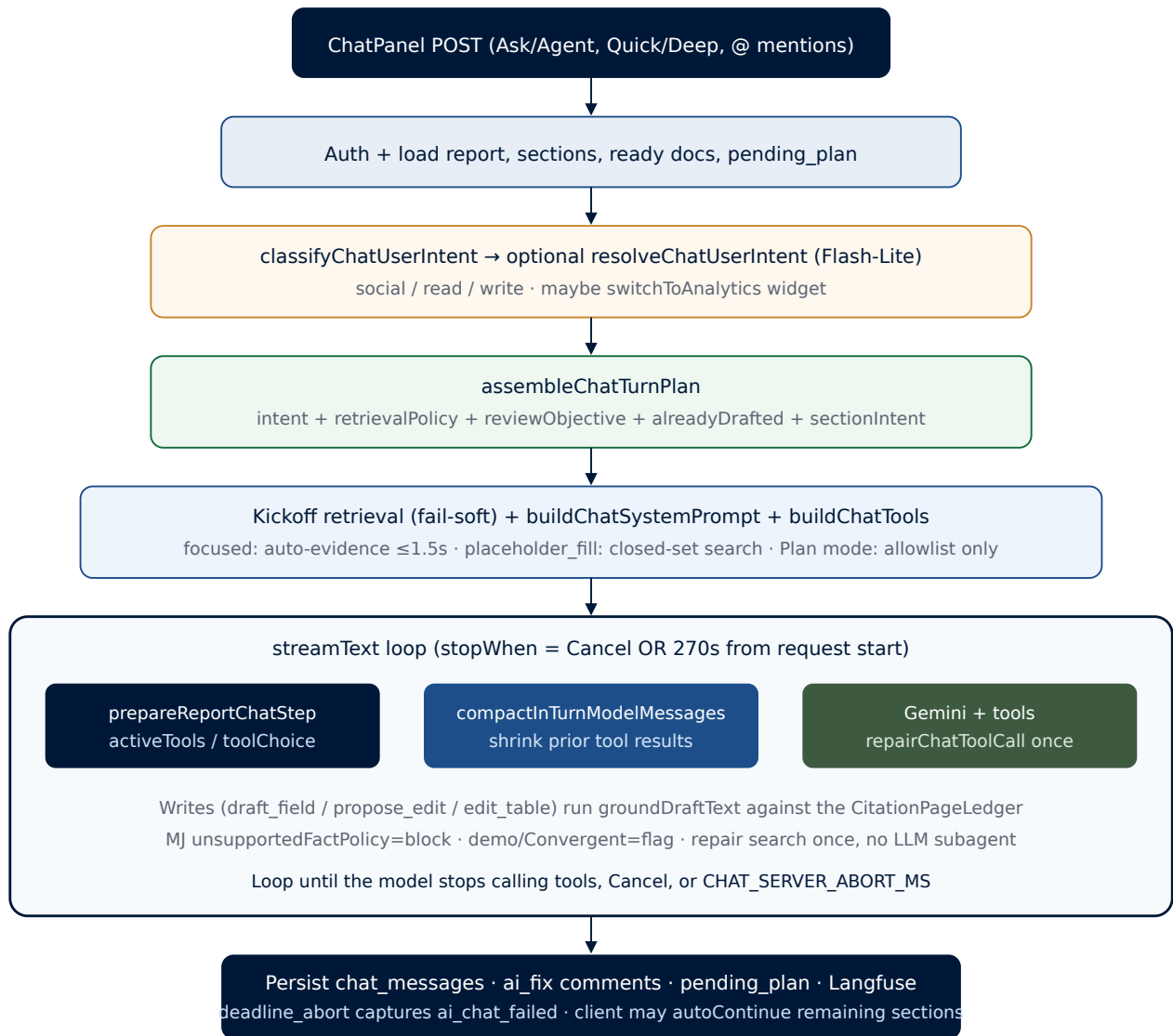


Figure 2. One POST = one turn. Cost is dominated by resending system + tool schemas + accumulated tool results on every step.

3. Where to read the code

Concern	Primary files
HTTP turn	src/app/api/reports/[reportId]/chat/route.ts
Intent	user-intent.ts, resolve-user-intent.ts
Turn snapshot	turn-plan.ts → ChatTurnPlan
Per-step tools	step-policy.ts → prepareReportChatStep
Retrieval policy	retrieval-policy.ts (focused adaptive comprehensive)
Indexing (write path)	run-document-ingest.ts, chunk-pages.ts, embed-chunks.ts — briefing 3
Hybrid search (read path)	retrieval.ts (pgvector + English FTS, RRF k=60) — briefing 4
Tool implementations	tools.ts (Plan allowlist in document-review.ts)
Page walk	document-review.ts, review-page-plan.ts
Search hide / ask_user hide	search-loop.ts
Closed-set blanks	placeholder-fill.ts
Fact gate	ground-draft.ts, citation-grounding.ts, claim-facts.ts
Payload size	compact-tool-history.ts, tool-result-budget.ts
Characterization lock	harness-scenarios.ts + colocated *.test.ts
Living plans	docs/harness-plan.md, docs/retrieval.md, docs/document-ingest-pipeline.md

Two control planes. The server hides tools in prepareStep. The prompt also tells the model not to use missing tools (intentToolAvailabilityRule). If you add a guard, pick one plane. Do not add a regex, a prompt paragraph, and a test for the same incident without retiring something. That is how this file grew.